

// for the developers running a table

NovaPay — technical facilitator briefing

NovaPay is a full-day workshop where UK students aged 13–18 debug a fake banking app and then build their own feature, both using GitHub Copilot. This doc skips the pedagogy and goes straight to how the thing actually works, what breaks, and how to fix it fast in front of a table of students.

THE STRUCTURE OF THE DAY

Mark runs the room — the clock, the slide decks, every phase transition. You own **one small table of students for the entire day**. That's the key thing to understand: you're not roaming backup, you're the only guide your table has, from the first bug to the final demo. When Mark moves the whole room from Brainstorm to Plan, it's you who actually gets your table there.

ACCOUNTS & SETUP — NOT YOUR JOB

Every student account, repo, and Codespace is already set up before you arrive. You won't provision anything. Know enough about the stack to troubleshoot (see below), but if an account is genuinely broken, that's an escalation to Mark, not something to fix from scratch mid-session.

Copy these onto your own laptop before the day starts — you'll be running your table on these, not looking things up live.

ANSWERS, FAST — THE ONES YOU'LL REACH FOR MOST

bug-answer-key.html	The private answer key for all 10 planted bugs — for you, not students	Bug Hunt
facilitator-reference.html	Bug answers plus strategy notes, including the Agent-mode 'fix all' warning	Bug Hunt + Build
printable-troubleshooting-card.html	Fast fixes for the common technical snags (push hangs, stale Codespace, etc.)	All day
printable-when-it-breaks.html	Composure-focused guide for total failures — wifi down, projector dies, you're behind schedule	All day

WHAT YOUR TABLE SEES — KNOW THESE COLD

participant-guide.html	The student-facing reference doc — know what's in it so you can point to it fast	All day
how-to-find-fix-bugs.html	The worked bug-fixing method your table follows before the real hunt	Bug Hunt
how-to-build-your-feature.html	The 10-step from-scratch tutorial your table follows together	Feature Tutorial

WORKING REFERENCES DURING THE BUILD

milestone-screen.html	Stretch goals by year group, in the right panel — for fast finishers at your table	Feature Creation
feature-test-plan.html	The ready-to-demo checklist — hand this to your table as they head into testing	Feature Creation
brainstorm-screen.html	The back-pocket idea gallery — your fallback when a group is stuck for an idea	Brainstorm

THIS DOCUMENT

novapay-developer-briefing.html	Everything else — the phase-by-phase guidance, bug diffs, and failure modes below	Read before the day
---	---	---------------------

IF YOU'RE GENUINELY STUCK

Flag Mark. He's running the room but is reachable — don't sit on a problem your table can't move past. (Worth confirming the exact signal for this with Mark before the day — a raised hand, a specific phrase, whatever's fastest to

02 The Stack

Context so you can help troubleshoot when something looks broken at your table. Every student: personal GitHub account → repo from a template → GitHub Codespace → VS Code in the browser, with GitHub Copilot and a custom instructions file.

NOT YOUR JOB

Account creation, repo templates, and Codespace provisioning are all set up in advance. If a student's account genuinely isn't working, that's an escalation, not something to debug from scratch on the day.

Personal GitHub account ▼

One per student. No org account — Business self-serve signup was paused, so personal accounts is the deliberate path, not a fallback.

- Free/Pro/Pro+ tier affects chat limits — see Accounts & Billing
- Each student's Copilot behaviour is fully isolated per-repo

Repo from template ▼

Created via 'Use this template'. Contains only what the student needs.

- bank-dashboard.html — the buggy app
- starter-template.html — optional reference
- challenge.js — pre-loaded buggy function for Level Up
- .github/copilot-instructions.md — Nova

GitHub Codespace ▼

Cloud dev environment, works on any device. This is where staleness bugs live.

- Pulls from the repo's last-pushed commit at creation time
- Won't see files added after creation until rebuilt
- 120 core-hours/month free on personal accounts — plenty for one day

VS Code + Live Server ▼

Live Server extension renders the HTML for immediate feedback.

- Correct extension ID: ritwickdey.LiveServer
- A wrong extension ID was an earlier bug — already fixed, but check devcontainer.json if issues resurface

Nova isn't a separate product — it's Copilot Chat with a repo-level `.github/copilot-instructions.md` that changes behaviour based on phase. GitHub Copilot reads this automatically for Chat, inline completions, and Agent mode in VS Code.

— RESTRICTIVE —

Instructions constrain Nova to **hints only** — it should ask leading questions ("what does this function return when the input's empty?") rather than hand over a fix. This is deliberate friction: the point is the debugging method, not the fixed file.

EXPECTED, NOT BROKEN

Students getting nudged rather than told the answer is the instructions working as designed. Don't "fix" this if a student complains Nova won't just tell them.

— GENERATIVE —

Instructions switch to generative — Nova builds with the student. **Agent mode specifically** (not Ask/Chat) is action-biased by default, which needed an explicit hard rule in the instructions file:

```
make ONE real, scoped fix, then STOP
and explain what you changed and why —
never refuse outright, never rewrite
everything in one pass
```

HISTORY HERE

The first version of this rule said "don't just fix everything" without also saying "but do fix something" — Nova over-corrected and started *refusing* to make any fix at all. If you're editing this file, test both directions: too eager AND too refusing.

— VERIFY —

The reliable check is the **References** list under a Copilot Chat response — expand it and confirm `.github/copilot-instructions.md` appears. Don't rely on judging tone or "does this feel restrictive" — that's subjective and slower to debug against.

If it's missing: the file may not exist at the exact path, the Codespace may not have pulled the latest commit (see Failure Modes), or the file was added after the Codespace was created and it needs a container rebuild.

The bug hunt and Level Up mostly run themselves — Nova and the slide decks do the heavy lifting. From the feature tutorial onward, the whole afternoon runs on how well you facilitate. This is the section that matters most. Click any phase.

10:00–10:30

Bug Tutorial + Bug Hunt

MOSTLY RUNS ITSELF ▲

YOUR ROLE

Circulate and watch, don't lecture. Nova is deliberately restrictive here — mirror that yourself. Nudge with questions, don't hand over the fix.

GOOD NUDGE QUESTIONS

- "What happens if you open the browser console — F12?"
- "Are there any bugs in this code?" (as a prompt for them to give Nova)
- "What does this look like compared to the working parts of the page?"

WATCH FOR

Groups asking Copilot to "fix everything at once" — it tends to miss bugs 3, 8, 9, and 10, since those are logic/data errors rather than syntax errors. Let it play out, then look together at what got missed. Good live lesson in why understanding beats blind trust.

Not a bug: Nova giving hints instead of answers during this phase is the instructions working as intended. Don't try to "fix" that if a student complains.

10:35–11:05

Level Up ⚡ (3 challenge rounds)

RUNS ITSELF ▲

YOUR ROLE

Keep pace tight — timers are built into the slide deck, so mostly you're clicking through and keeping energy up. Fix the Bug (7 min) uses `challenge.js`, already pre-loaded in every repo — don't let anyone waste time manually creating that file.

ROUNDS

- **Fix the Bug** — Copilot fixes a buggy function, proved with Node in the terminal
- **Comment Race** — write a comment, Tab-complete, first working function wins
- **Best Prompt** — one Chat message only, best result wins

11:20–11:40

Feature Tutorial — everyone together

GUIDED, LOW-STAKES



WHAT'S HAPPENING

Everyone builds a Savings Tracker from a genuinely blank file, together, following [how-to-build-your-feature.html](#) step by step. This is the technique they'll reuse on their own idea straight after — skeleton, colours, header, form, progress bar, logic, wiring.

YOUR ROLE

Help stragglers catch up to the group pace rather than letting them fall behind — this is meant to be followed together, not independently. Y11+ have exact comments hidden behind a reveal button; encourage them to try writing their own comment before revealing it.

11:40–11:55

Brainstorm

FACILITATION STARTS MATTERING



WHAT'S HAPPENING

Students pick their own feature idea — deliberately different from the Savings Tracker they just built together.

YOUR ROLE — THE ACHIEVABILITY CHECK

For every group's pick, ask: *"Can you describe it in one sentence? Does it have inputs, a calculation, and an output?"* If yes, approve it. If they're struggling to answer, the idea isn't scoped yet — help them narrow it before they move on.

SCOPE VETO PHRASES — USE THESE OFTEN

Anything needing a login, real accounts, a database, or "like the real app but..." needs redirecting:

"Brilliant idea for version 2. What's the one-screen version of it you could demo today?"

IF A GROUP IS STUCK FOR AN IDEA

Point at the back-pocket gallery on [brainstorm-screen.html](#) slide 2 — 7 pre-vetted features with difficulty labels. Just not the Savings Tracker, they've already built that one.

11:55–12:05pm

Plan on Paper

ONE THING TO CHECK



WHAT'S HAPPENING

Keyboards down, paper only. Four questions: what does it do, what does the user see, what are the inputs, what does it output.

YOUR ROLE — THE ONE CHECK THAT MATTERS

Circulate and look at every sketch. The single thing to verify: **are the inputs and outputs named?** ("goalName, targetAmount" — not "some boxes"). Named inputs become good Copilot prompts a few minutes later. Vague ones lead to vague, unproductive prompting.



WHY THIS PHASE CARRIES THE DAY

This is the single largest block and the one with no built-in scaffolding — no slide deck driving it, no timer telling students what to do next. It runs on how well you facilitate. Everything else in the day exists to prepare students for these 85 minutes.

THE METHOD TO REINFORCE (SAY THIS EARLY, REFER BACK TO IT CONSTANTLY)

"One: write a comment describing the next small thing. Two: let Copilot suggest, press Tab. Three: save, check Live Server. Four: if it's wrong, ask Nova why. Small steps — comment, code, check. Never write more than one piece before testing it."

ROUGH INTERNAL PACING (NOTHING IS ANNOUNCED OR ENFORCED — THIS IS JUST YOUR OWN SENSE OF WHERE GROUPS SHOULD BE)

- **~25 min in:** heading exists, inputs exist, a button exists, opens in Live Server without errors. Ugly is fine.
- **~60 min in:** styled, basic JS working — real inputs produce a real output on screen.
- **~75 min in:** should be shifting into testing — see below.

PROMPTING TIPS TO ACTIVELY REINFORCE

- "Write a function that takes goalAmount and savedAmount and returns the percentage complete"
- "Add a progress bar that fills based on the percentage variable"
- "Make a savings tracker" — too vague, will produce something generic and hard to build on
- "Fix my code" — no context, Copilot can't help without specifics

HELP STYLE BY YEAR GROUP

- **Y9–10:** concrete debugging questions — "why isn't my progress bar changing width?"
- **Y11:** can handle "review this function for edge cases I might have missed"
- **Y12–13:** can handle "how would you refactor this to be more maintainable?"

THE HARD SCOPE CALL — USE IT ONCE, TO THE WHOLE ROOM

If any group has nothing working by roughly the two-thirds mark, don't single them out — say this to everyone, so the stuck group hears it without being called out:

"Strip it back. One input, one button, one number on screen. Get that working, then add back."

TESTING IS FOLDED INTO THIS BLOCK — THERE'S NO SEPARATE SLOT

As groups near the end, proactively nudge rather than waiting for an announcement: *"Code freeze soon — try to break your own feature on purpose. Empty fields, negative numbers, huge numbers."* `feature-test-plan.html` has a ready-to-demo checklist if a group wants structure for this.

STRETCH GOALS FOR GROUPS THAT FINISH EARLY

`milestone-screen.html`'s right panel has stretch goals by year group. Generic ones: save to `localStorage` so it persists on refresh, support multiple goals, add a date field with a weekly-saving calculation, add a "simulate a transfer" button.

DEBUGGING THEIR OWN BUGS (DIFFERENT FROM THE PLANTED 10)

These are genuinely open-ended — whatever a student introduces. Common patterns worth knowing: a script tag not linked to the HTML file, a mismatched id between HTML and `getElementById`, a missing quote or bracket breaking everything below it, or a function defined but never called. Teach the "read the error message, don't just reread the code" habit — the console (F12) usually names the exact line.

2:05–2:20pm

Show & Tell

KEEP IT MOVING



YOUR ROLE

2 minutes per group. After each demo, ask the room "what did you like about that?" — keeps energy up and models the tone you want.

IF A DEMO BREAKS LIVE

"Congratulations — a live demo failure. That's the most authentic developer experience of the day."

Then let them show it working in the editor instead. Never let a broken demo become an awkward silence.

2:20–2:30pm

Feedback

LIGHT TOUCH



YOUR ROLE

Keep it short — energy is naturally dropping after demos. Help keep the round moving quickly rather than let it drag; this isn't the moment for deep reflection.

Not the full ~46-file pack — just the ones you'll actually open, project, or point students at during a live session.

BUG HUNT PHASE

how-to-find-fix-bugs.html	The worked, fake-example walkthrough — projected or opened together before the real hunt	10:00–10:10
bug-hunt-screen.html	The bug hunt brief and hint prompts — projected, left up the whole hunt	10:10–10:30
bug-test-plan.html	Where students log each bug found and how they fixed it	10:10–10:30

LEVEL UP

level-up-screen.html	13-slide deck with built-in round timers — click through, don't narrate the timing yourself	10:35–11:05
challenge.js	Pre-loaded buggy function in every student repo — Round 1 works on this, never create it fresh	10:35–11:05

FEATURE TUTORIAL

how-to-build-your-feature.html	The 10-step from-scratch Savings Tracker build, followed together	11:20–11:40
practice-feature.html	The file students build during the tutorial — later copied from for styling their own feature	11:20–11:40

BRAINSTORM & PLAN

brainstorm-screen.html	Popcorn round, categories, and the back-pocket idea gallery on slide 2	11:40–11:55
--	--	-------------

FEATURE CREATION

milestone-screen.html	Projector reference during the build — has stretch goals by year group in its right panel	12:40–2:05pm
feature-test-plan.html	Ready-to-demo checklist — point groups here as they head into testing near the end	12:40–2:05pm
participant-guide.html	Student's own reference doc if they lose track of what they're doing	All day

NOVA'S BRAIN

06 The 10 Planted Bugs — Technical Detail

All in `bank-dashboard.html`. This is the version with the actual diffs, for when you need to point at the exact line fast.

#	TYPE	BUG → FIX
1	Markup	Missing <code></div></code> after the balance footer — add it back
2	CSS	<code>color: #0F1340</code> → <code>color: #FFFFFF</code>
3	Data	Coffee transaction: <code>type: "credit"</code> → <code>type: "debit"</code>
4	Markup	<code>type="text"</code> → <code>type="number"</code>
5	JS/typo	<code>onlick=</code> → <code>onclick=</code>
6	Logic	<code>balance + amount</code> → <code>balance - amount</code> on withdrawal
7	JS/typo	<code>"balence-display"</code> → <code>"balance-display"</code> — silent <code>getElementById</code> null
8	Logic	Add <code>isNaN()</code> guard before calculating — empty input currently produces <code>NaN</code>
9	Logic	<code>*rate</code> → <code>*(rate / 100)</code> — interest formula off by 100x
10	Logic	Add <code>if (amount > balance) return</code> guard — no overdraft check exists

BUGS 3, 8, 9, 10 ARE THE ONES COPILOT TENDS TO MISS

They're logic/data errors, not syntax errors, so "ask Copilot to fix everything" often skips them. If a group tries this, let it play out, then look together at what got missed and why — a live lesson in why understanding beats blind trust.

Everything here came out of real testing, not theory. Ordered roughly by how often you'll hit them.

HIGH

Save vs push — the #1 cause of 'my fix disappeared'**Symptom**

Student swears they fixed something, Codespace still shows the bug, or a new Codespace doesn't have their earlier work.

Cause

Ctrl+S only saves to the Codespace's own disk. It does NOT push to GitHub. A new Codespace always pulls from the last *pushed* commit — so unpushed work is invisible to any other Codespace and lost entirely if that Codespace is deleted.

Fix

Check Source Control panel for uncommitted/unpushed changes. Commit + push. This single distinction accounts for most "Nova still ignores my fix" reports.

HIGH

Codespace staleness**Symptom**

A file you know is in the repo (e.g. copilot-instructions.md) doesn't seem to affect Copilot's behaviour.

Cause

If the Codespace was created before that file was pushed, it won't see it until the container rebuilds.

Fix

Ctrl+Shift+P → "Rebuild Container", or delete and recreate the Codespace (repo files are safe either way).

MED

git push hangs or times out**Symptom**

Push spins indefinitely from inside the Codespace terminal or Source Control panel.

Fix

Bypass the Codespace's network path entirely — copy the file content, paste into github.com's own web-based file editor, and commit there. Uses the browser's connection, not the Codespace's.

If it keeps happening

Run `git push -v` and `gh auth status` for diagnostics, but the web-editor workaround is faster mid-session than debugging network config live.

MED

False-positive 'SQL injection' refusal on pasted code**Symptom**

Copilot Chat refuses to help with a loan-calculation or similar numeric function, citing security concerns.

Cause

Cold-pasting a snippet with no file context sometimes trips a false positive.

Fix already applied

challenge.js is pre-loaded as a real repo file rather than typed fresh into chat — having real file context avoids the false positive. If it recurs elsewhere, the same fix (make it a real file, not a paste) applies.

MED**Nova refuses to fix anything in Agent mode****Symptom**

During the build phase, Agent mode won't make any changes at all, even trivial ones.

Cause

An overly strict "don't fix everything at once" instruction can overcorrect into "don't fix anything."

Fix

The instructions file should say: make ONE real fix, then stop and explain. If you're troubleshooting a specific student's repo, check their copilot-instructions.md matches the current version — not an earlier overcorrected draft.

LOW**One student's Nova is oddly restrictive during the build phase****Symptom**

A single student's Copilot Chat behaves like bug-hunt-phase Nova during feature building.

Cause

Per-repo instructions, so this is isolated to that one student — not a systemic issue.

Fix

Rename their .github/copilot-instructions.md on github.com (removes it from consideration), rebuild their Codespace. Copilot reverts to normal default behaviour for them only.

08**Accounts & Billing**

Worth knowing if a student or co-facilitator asks why Copilot "stopped working" mid-session.

COPILOT FREE — 50 CHAT MESSAGES/MONTH

As of GitHub's June 2026 usage-based billing shift, personal accounts on the Free tier get 2,000 code completions but only **50 chat requests a month**. A day this chat-heavy (bug hints, three Level Up rounds, 85 minutes of iterative building) can burn through that before lunch on a brand-new free account. If students are on **paid Pro/Pro+ seats**, this doesn't apply — but paid tiers now run on an AI Credit allowance too (\$10 Pro = \$10/month in credits), not literally unlimited, so heavy Agent-mode use across a full day is still worth a mental note.

GITHUB EDUCATION STUDENT ACCOUNTS

A different path: unlimited code completions, but chat/agent usage is still capped, and — since June 24, 2026 — Free and Student tiers use **auto model-selection only**, no manual model picker. Don't build a live demo around picking a specific model on these accounts.

For fast diagnosis while circulating. Roughly ordered by which phase of the day you'll see it in.

Button does nothing when clicked

Check for onclick vs onclick typo (Bug 5), or check browser console (F12) for a JS error

Balance shows but looks blank

Check text colour against background — Bug 2, invisible text is a colour match, not missing text

Number shows as NaN

Missing isNaN() guard on empty input — Bug 8, or check if a similar guard is needed in a student's own feature

'Are there bugs in this code?' gets a vague answer

Expected — Nova is restrictive by design in this phase. Push toward hint prompts like the ones on bug-hunt-screen.html rather than open questions

Copilot Chat won't help with a pasted function

Likely a false-positive content flag on cold-pasted code — have them work in a real file instead of pasting into chat directly

Student's fix 'disappeared' after reopening

They saved but didn't commit+push — see Failure Modes above, this is the most common one

New Codespace doesn't reflect a recent repo change

Rebuild container, or recreate — it pulled from an earlier pushed state

Feature idea keeps growing scope (login, database, 'like the real app')

Redirect: 'Brilliant idea for version 2 — what's the one-screen version you could demo today?'

Group has nothing working by the build phase midpoint

Hard scope call, said to the whole room: 'Strip it back — one input, one button, one number on screen. Get that working, then add back.'

Run this the night before or morning of, on a throwaway test repo.

- ☐ Upload the current copilot-instructions.md + challenge.js to a throwaway test repo
- ☐ Open a Codespace on it and confirm the References list on a Chat response shows the instructions file
- ☐ Deliberately try to break Agent mode both ways: does it refuse everything? Does it rewrite everything in one pass? It should do neither
- ☐ Confirm the Live Server extension installs correctly via devcontainer.json (ritwickdey.LiveServer)
- ☐ Run the critical path once end to end: create account → Codespace → Live Server → fix one bug → three steps of the feature tutorial
- ☐ Check which Copilot tier the student accounts are actually on before relying on heavy chat usage all day
- ☐ Confirm build-phase-guide.html's status — it's currently out of step with the rest of the pack, decide fix/retire/ignore